

MERGE SORT USING DIVIDE AND CONQUER

Aim

To implement Merge Sort algorithm using the Divide and Conquer approach.

Algorithm

1. Read the size of the array and its elements.
2. Divide the array into two halves recursively until a single element remains.
3. Find the middle position of each subarray.
4. Compare elements from both halves and merge them in sorted order.
5. Continue merging all subarrays until the complete array is sorted.
6. Display the original array and the sorted array.

Program

```
import java.util.Arrays;
import java.util.Scanner;

public class MergeSort {
    static void merge(int[] a, int l, int m, int r) {
        int[] b = new int[r - l + 1]; int i = l, j = m + 1, k = 0;
        while (i <= m && j <= r) b[k++] = a[i] <= a[j] ? a[i++] : a[j++];
        while (i <= m) b[k++] = a[i++];
        while (j <= r) b[k++] = a[j++];
        for (int x = 0; x < k; x++) a[l + x] = b[x];
    }
    static void sort(int[] a, int l, int r) {
        if (l < r) { int m = l + (r - l) / 2; sort(a, l, m); sort(a, m + 1, r); merge(a, l, m, r); }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size: ");
        int n = sc.nextInt();
        int[] a = new int[n];
        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        System.out.println("Original array: " + Arrays.toString(a));
        sort(a, 0, n - 1);
        System.out.println("Sorted array: " + Arrays.toString(a));
    }
}
```

Output

```
Enter size: 8
Enter 8 elements:
12
5
9
17
2
15
8
6
Original array: [12, 5, 9, 17, 2, 15, 8, 6]
Sorted array: [2, 5, 6, 8, 9, 12, 15, 17]
```

Result

Thus, the program to implement the Merge Sort algorithm using the Divide and Conquer approach has been executed successfully, and the given array was sorted in ascending order.